Tema 3: Clases Anidadas

Pablo Gordillo (pabgordi@ucm.es)

Departamento de Sistemas Informáticos y Computación

Facultad de Informática



(Basado en el material creado por Puri Arenas, Jesús Correas y Samir Genaim)

Introducción

- **Anidar una clase**: Definir una clase dentro de otra clase o interfaz.
- Las clases anidadas se pueden dividir en:
  - Estáticas

```
class ClaseExterna {
  ...
  static class NestedClass { // clase anidada
    ...
  }
  ...
}
```

  - No Estáticas

```
class ClaseExterna {
  ...
  class NestedClass { // clase interna (inner class)
    ...
  }
  ...
}
```

Introducción

- Una clase anidada en realidad es un atributo de la clase externa, y por tanto va acompañado de un cualificador: **public, protected, private**.

Introducción

- Una clase anidada en realidad es un atributo de la clase externa, y por tanto va acompañado de un cualificador: **public, protected, private**.
- Cuando se compila ClaseExterna se generan los ficheros **ClaseExterna.class** y **ClaseExterna$NestedClass.class**

Introducción

- Una clase anidada en realidad es un atributo de la clase externa, y por tanto va acompañado de un cualificador: **public, protected, private**.
- Cuando se compila ClaseExterna se generan los ficheros **ClaseExterna.class** y **ClaseExterna$NestedClass.class**
- El uso de clases anidadas permite:
  - **Agrupar las clases de forma lógica**: Si una clase es útil solamente a otra clase, entonces es lógico anidarla en esa clase y mantenerlas juntas. Si una clase B solo va a ser utilizada por otra clase A (es una clase auxiliar) podemos definir B dentro de A. El anidamiento tal como "clases de ayuda" hace sus paquetes más uniformes.
  - **Aumentar el nivel de encapsulación**: La clase interna puede ser privada, pero tiene acceso a todos los atributos (públicos, privados o protegidos) de la clase externa.
  - **Generar un código más legible y de fácil mantenimiento**: Las clases anidadas suelen ser pequeñas, y al estar dentro de la clase que las usa permite aumentar la legibilidad del código.

Clases anidadas estáticas

- Si una clase anidada posee un atributo (no constante) estático, entonces debemos marcarla como clase estática.

Clases anidadas estáticas

- Si una clase anidada posee un atributo (no constante) estático, entonces debemos marcarla como clase estática.
- El cualificador (**public, protected, private**) de una clase estática funciona igual que el cualificador de un atributo.

Clases anidadas estáticas

- Si una clase anidada posee un atributo (no constante) estático, entonces debemos marcarla como clase estática.
- El cualificador (**public, protected, private**) de una clase estática funciona igual que el cualificador de un atributo.
- Como en el caso de métodos y atributos, una clase estática interna es miembro de su clase externa, así que tiene acceso a sus atributos y métodos privados.

Clases anidadas estáticas

- Si una clase anidada posee un atributo (no constante) estático, entonces debemos marcarla como clase estática.
- El cualificador (**public, protected, private**) de una clase estática funciona igual que el cualificador de un atributo.
- Como en el caso de métodos y atributos, una clase estática interna es miembro de su clase externa, así que tiene acceso a sus atributos y métodos privados.
- Como los métodos estáticos, una clase **estática interna** no tiene acceso directo a los atributos no-estáticos de la clase externa (necesitamos una instancia para poder acceder) **[a diferencia de internas no-estáticas]**
- Usar los métodos y atributos de la clase interna depende de donde accedamos (tienen que ser visibles)

Clases anidadas estáticas

```
public class OuterClass {
  static private int outerA;
  private int outerB;

  public static class NestedClass {
    static private int nestedA;
    private int nestedB;

    public void nestedMethod(){
      OuterClass.outerA = 3;
      OuterClass.outerB = 15;
      OuterClass o1 = new OuterClass();
      o1.outerB = 3;
      o1.outerMethod();
    }
  }

  public void outerMethod(){
    NestedClass.nestedA = 3;
    NestedClass.nestedB = 12;
    NestedClass o2 = new NestedClass();
    o2.nestedB = 10;
    o2.nestedMethod();
  }
}
```

Clases anidadas estáticas

```
public class OuterClass {
  static private int outerA;
  private int outerB;

  public static class NestedClass {
    static private int nestedA; //obliga a que la clase sea estática
    private int nestedB;

    public void nestedMethod(){
      OuterClass.outerA = 3;
      OuterClass.outerB = 15;
      OuterClass o1 = new OuterClass();
      o1.outerB = 3;
      o1.outerMethod();
    }
  }

  public void outerMethod(){
    NestedClass.nestedA = 3;
    NestedClass.nestedB = 12;
    NestedClass o2 = new NestedClass();
    o2.nestedB = 10;
    o2.nestedMethod();
  }
}
```

Clases anidadas estáticas

```
public class OuterClass {
  static private int outerA;
  private int outerB;

  public static class NestedClass {
    static private int nestedA;
    private int nestedB;

    public void nestedMethod(){
      OuterClass.outerA = 3;          //?
      OuterClass.outerB = 15;
      OuterClass o1 = new OuterClass();
      o1.outerB = 3;
      o1.outerMethod();
    }
  }

  public void outerMethod(){
    NestedClass.nestedA = 3;
    NestedClass.nestedB = 12;
    NestedClass o2 = new NestedClass();
    o2.nestedB = 10;
    o2.nestedMethod();
  }
}
```

Clases anidadas estáticas

```
public class OuterClass {
  static private int outerA;
  private int outerB;

  public static class NestedClass {
    static private int nestedA;
    private int nestedB;

    public void nestedMethod(){
      OuterClass.outerA = 3;          //OK
      OuterClass.outerB = 15;         //?
      OuterClass o1 = new OuterClass();
      o1.outerB = 3;
      o1.outerMethod();
    }
  }

  public void outerMethod(){
    NestedClass.nestedA = 3;
    NestedClass.nestedB = 12;
    NestedClass o2 = new NestedClass();
    o2.nestedB = 10;
    o2.nestedMethod();
  }
}
```

Clases anidadas estáticas

```
public class OuterClass {
  static private int outerA;
  private int outerB;

  public static class NestedClass {
    static private int nestedA;
    private int nestedB;

    public void nestedMethod(){
      OuterClass.outerA = 3;          //OK
      OuterClass.outerB = 15;         //WRONG-No Estático
      OuterClass o1 = new OuterClass();
      o1.outerB = 3;                  //?
      o1.outerMethod();
    }
  }

  public void outerMethod(){
    NestedClass.nestedA = 3;
    NestedClass.nestedB = 12;
    NestedClass o2 = new NestedClass();
    o2.nestedB = 10;
    o2.nestedMethod();
  }
}
```

Clases anidadas estáticas

```
public class OuterClass {
  static private int outerA;
  private int outerB;

  public static class NestedClass {
    static private int nestedA;
    private int nestedB;

    public void nestedMethod(){
      OuterClass.outerA = 3;          //OK
      OuterClass.outerB = 15;         //WRONG-No Estático
      OuterClass o1 = new OuterClass();
      o1.outerB = 3;                  //OK
      o1.outerMethod();               //?
    }
  }

  public void outerMethod(){
    NestedClass.nestedA = 3;
    NestedClass.nestedB = 12;
    NestedClass o2 = new NestedClass();
    o2.nestedB = 10;
    o2.nestedMethod();
  }
}
```

Clases anidadas estáticas

```
public class OuterClass {
  static private int outerA;
  private int outerB;

  public static class NestedClass {
    static private int nestedA;
    private int nestedB;

    public void nestedMethod(){
      OuterClass.outerA = 3;          //OK
      OuterClass.outerB = 15;         //WRONG-No Estático
      OuterClass o1 = new OuterClass();
      o1.outerB = 3;                  //OK
      o1.outerMethod();               //OK
    }
  }

  public void outerMethod(){
    NestedClass.nestedA = 3;          //?
    NestedClass.nestedB = 12;
    NestedClass o2 = new NestedClass();
    o2.nestedB = 10;
    o2.nestedMethod();
  }
}
```

Clases anidadas estáticas

```
public class OuterClass {
  static private int outerA;
  private int outerB;

  public static class NestedClass {
    static private int nestedA;
    private int nestedB;

    public void nestedMethod(){
      OuterClass.outerA = 3;          //OK
      OuterClass.outerB = 15;         //WRONG-No Estático
      OuterClass o1 = new OuterClass();
      o1.outerB = 3;                  //OK
      o1.outerMethod();               //OK
    }
  }

  public void outerMethod(){
    NestedClass.nestedA = 3;          //OK
    NestedClass.nestedB = 12;         //?
    NestedClass o2 = new NestedClass();
    o2.nestedB = 10;
    o2.nestedMethod();
  }
}
```

Clases anidadas estáticas

```
public class OuterClass {
  static private int outerA;
  private int outerB;

  public static class NestedClass {
    static private int nestedA;
    private int nestedB;

    public void nestedMethod(){
      OuterClass.outerA = 3;          //OK
      OuterClass.outerB = 15;         //WRONG-No Estático
      OuterClass o1 = new OuterClass();
      o1.outerB = 3;                  //OK
      o1.outerMethod();               //OK
    }
  }

  public void outerMethod(){
    NestedClass.nestedA = 3;          //OK
    NestedClass.nestedB = 12;         //WRONG-No Estático
    NestedClass o2 = new NestedClass();
    o2.nestedB = 10;                  //?
    o2.nestedMethod();
  }
}
```

Clases anidadas estáticas

```
public class OuterClass {
  static private int outerA;
  private int outerB;

  public static class NestedClass {
    static private int nestedA;
    private int nestedB;

    public void nestedMethod(){
      OuterClass.outerA = 3;          //OK
      OuterClass.outerB = 15;         //WRONG-No Estático
      OuterClass o1 = new OuterClass();
      o1.outerB = 3;                  //OK
      o1.outerMethod();               //OK
    }
  }

  public void outerMethod(){
    NestedClass.nestedA = 3;          //OK
    NestedClass.nestedB = 12;         //WRONG-No Estático
    NestedClass o2 = new NestedClass();
    o2.nestedB = 10;                  //OK
    o2.nestedMethod();                //?
  }
}
```

Clases anidadas estáticas

```
public class OuterClass {
  static private int outerA;
  private int outerB;

  public static class NestedClass {
    static private int nestedA;
    private int nestedB;

    public void nestedMethod(){
      OuterClass.outerA = 3;          //OK
      OuterClass.outerB = 15;         //WRONG-No Estático
      OuterClass o1 = new OuterClass();
      o1.outerB = 3;                  //OK
      o1.outerMethod();               //OK
    }
  }

  public void outerMethod(){
    NestedClass.nestedA = 3;          //OK
    NestedClass.nestedB = 12;         //WRONG-No Estático
    NestedClass o2 = new NestedClass();
    o2.nestedB = 10;                  //OK
    o2.nestedMethod();                //OK
  }
}
```

Múltiples niveles de anidación

- Podemos crear varios **niveles de anidación** teniendo en cuenta **que una clase estática no puede estar dentro de una clase no estática** (a no ser que la clase no estática sea la clase raíz).

```
public class OuterClass {
  public static int f1;
  protected int f3;

  static class NestedClass1 {
    private static int f2;

    private static class NestedClass2 {
      private void moo() {
        OuterClass.f1 = 1;                    //??
        OuterClass.NestedClass1.f2 = 1;
        f3 = 1;
      }
    }
  }
}
```

Múltiples niveles de anidación

- Podemos crear varios **niveles de anidación** teniendo en cuenta **que una clase estática no puede estar dentro de una clase no estática** (a no ser que la clase no estática sea la clase raíz).

```
public class OuterClass {
  public static int f1;
  protected int f3;

  static class NestedClass1 {
    private static int f2;

    private static class NestedClass2 {
      private void moo() {
        OuterClass.f1 = 1;                    //OK
        OuterClass.NestedClass1.f2 = 1;       //??
        f3 = 1;
      }
    }
  }
}
```

Múltiples niveles de anidación

- Podemos crear varios **niveles de anidación** teniendo en cuenta **que una clase estática no puede estar dentro de una clase no estática** (a no ser que la clase no estática sea la clase raíz).

```
public class OuterClass {
  public static int f1;
  protected int f3;

  static class NestedClass1 {
    private static int f2;

    private static class NestedClass2 {
      private void moo() {
        OuterClass.f1 = 1;                    //OK
        OuterClass.NestedClass1.f2 = 1;       //OK
        f3 = 1;                               //??
      }
    }
  }
}
```

Múltiples niveles de anidación

- Podemos crear varios **niveles de anidación** teniendo en cuenta **que una clase estática no puede estar dentro de una clase no estática** (a no ser que la clase no estática sea la clase raíz).

```
public class OuterClass {
  public static int f1;
  protected int f3;

  static class NestedClass1 {
    private static int f2;

    private static class NestedClass2 {
      private void moo() {
        OuterClass.f1 = 1;                    //OK
        OuterClass.NestedClass1.f2 = 1;       //OK
        f3 = 1;                               //WRONG - f3 no es de NestedClass2
      }
    }
  }
}
```

Acceso a una clase anidada esática

```
public class OuterClass {
  static class NestedClass1 {
    public static int f1;
    private int f4;

    static class NestedClass2 {
      private static int f2;
      public int f3;
    }
  }

  void foo() {
    NestedClass1.f1 = 1;                          //??
    NestedClass1.NestedClass2.f2 = 1;
    NestedClass1.NestedClass2.f3 = 1;
    NestedClass1.f4 = 1;
  }
}
```

- OuterClass puede acceder a todos los atributos y métodos estáticos de NestedClass usando el nombre como prefijo.

Acceso a una clase anidada esática

```
public class OuterClass {
  static class NestedClass1 {
    public static int f1;
    private int f4;

    static class NestedClass2 {
      private static int f2;
      public int f3;
    }
  }

  void foo() {
    NestedClass1.f1 = 1;                    //OK
    NestedClass1.NestedClass2.f2 = 1;       //??
    NestedClass1.NestedClass2.f3 = 1;
    NestedClass1.f4 = 1;
  }
}
```

- OuterClass puede acceder a todos los atributos y métodos estáticos de NestedClass usando el nombre como prefijo.

Acceso a una clase anidada esática

```
public class OuterClass {
  static class NestedClass1 {
    public static int f1;
    private int f4;

    static class NestedClass2 {
      private static int f2;
      public int f3;
    }
  }

  void foo() {
    NestedClass1.f1 = 1;                    //OK
    NestedClass1.NestedClass2.f2 = 1;       //OK
    NestedClass1.NestedClass2.f3 = 1;       //??
    NestedClass1.f4 = 1;
  }
}
```

- OuterClass puede acceder a todos los atributos y métodos estáticos de NestedClass usando el nombre como prefijo.

Acceso a una clase anidada esática

```
public class OuterClass {
  static class NestedClass1 {
    public static int f1;
    private int f4;

    static class NestedClass2 {
      private static int f2;
      public int f3;
    }
  }

  void foo() {
    NestedClass1.f1 = 1;                    //OK
    NestedClass1.NestedClass2.f2 = 1;       //OK
    NestedClass1.NestedClass2.f3 = 1;       //WRONG - f3 no es estático
    NestedClass1.f4 = 1;                    //??
  }
}
```

- OuterClass puede acceder a todos los atributos y métodos estáticos de NestedClass usando el nombre como prefijo.

Acceso a una clase anidada esática

```
public class OuterClass {
  static class NestedClass1 {
    public static int f1;
    private int f4;

    static class NestedClass2 {
      private static int f2;
      public int f3;
    }
  }

  void foo() {
    NestedClass1.f1 = 1;                    //OK
    NestedClass1.NestedClass2.f2 = 1;       //OK
    NestedClass1.NestedClass2.f3 = 1;       //WRONG - f3 no es estático
    NestedClass1.f4 = 1;                    //WRONG - f4 no es estático
  }
}
```

- OuterClass puede acceder a todos los atributos y métodos estáticos de NestedClass usando el nombre como prefijo.

Acceso a una clase anidada desde otra clase

Si en una clase **A** accedemos a $\mathbf{C_1.C_2.C_3...C_n.f}$ entonces **f** y todas las clases $\mathbf{C_1}$ tienen que ser visibles a **A**

1. Si **A** y $\mathbf{C_1}$ están en paquetes distintos, entonces todas las clases $\mathbf{C_i}$ y **f** tienen que ser **public**.
2. Si **A** y $\mathbf{C_1}$ están en el mismo paquete, entonces todas las clases $\mathbf{C_i}$ y **f** tienen que ser **public, protected** o **package private**.

Lo mismo se aplica si llamamos a un método $\mathbf{C_1.C_2.C_3...C_n.m}$

Ex.1 - es Válido?

```
package tests;

public class C1 {
  static public class C2 {
    static public class C3 {
      static public class C4 {
        static public int f;
      }
    }
  }
}
```

```
package tests;

public class A {
  void m() {
    C1.C2.C3.C4.f = 1;
  }
}
```

Las reglas de acceso a los métodos y atributos de una clase interna es similar a las de una clase estándar, asumiendo que la clase interna se comporta como un atributo más.

Ex.1 - es Válido?

```
package tests;

public class C1 {
  static public class C2 {
    static public class C3 {
      static public class C4 {
        static public int f;
      }
    }
  }
}
```

```
package tests;

public class A {
  void m() {
    C1.C2.C3.C4.f = 1;
  }
}
```

OK!

Las reglas de acceso a los métodos y atributos de una clase interna es similar a las de una clase estándar, asumiendo que la clase interna se comporta como un atributo más.

Ex.2 - es Válido?

```
package tests;

public class C1 {
  static public class C2 {
    static private class C3 {
      static public class C4 {
        static public int f;
      }
    }
  }
}
```

```
package tests;

public class A {
  void m() {
    C1.C2.C3.C4.f = 1;
  }
}
```

Ex.2 - es Válido?

```
package tests;

public class C1 {
  static public class C2 {
    static private class C3 {
      static public class C4 {
        static public int f;
      }
    }
  }
}
```

```
package tests;

public class A {
  void m() {
    C1.C2.C3.C4.f = 1;
  }
}
```

ERROR! C3 es privada

Ex.3 - es Válido?

```
package tests;

public class C1 {
  static public class C2 {
    static protected class C3 {
      static public class C4 {
        static public int f;
      }
    }
  }
}
```

```
package tests;

public class A {
  void m() {
    C1.C2.C3.C4.f = 1;
  }
}
```

Ex.3 - es Válido?

```
package tests;

public class C1 {
  static public class C2 {
    static protected class C3 {
      static public class C4 {
        static public int f;
      }
    }
  }
}
```

```
package tests;

public class A {
  void m() {
    C1.C2.C3.C4.f = 1;
  }
}
```

OK! C3 es protected y están en el mismo paquete

Ex.4 - es Válido?

```
package tests;

public class C1 {
  static public class C2 {
    static class C3 {
      static public class C4 {
        static public int f;
      }
    }
  }
}
```

```
package tests;

public class A {
  void m() {
    C1.C2.C3.C4.f = 1;
  }
}
```

Ex.4 - es Válido?

```
package tests;

public class C1 {
  static public class C2 {
    static class C3 {
      static public class C4 {
        static public int f;
      }
    }
  }
}
```

```
package tests;

public class A {
  void m() {
    C1.C2.C3.C4.f = 1;
  }
}
```

OK! C3 es package private y están en el mismo paquete

Ex.5 - es Válido?

```
package first ;

public class C1 {
  static public class C2 {
    static public class C3 {
      static public class C4 {
        static public int f;
      }
    }
  }
}
```

```
package second;

public class A {
  void m() {
    C1.C2.C3.C4.f = 1;
  }
}
```

Ex.5 - es Válido?

```
package first ;

public class C1 {
  static public class C2 {
    static public class C3 {
      static public class C4 {
        static public int f;
      }
    }
  }
}
```

```
package second;

public class A {
  void m() {
    C1.C2.C3.C4.f = 1;
  }
}
```

OK!

Ex.6 - es Válido?

```
package first ;

public class C1 {
  static public class C2 {
    static private class C3 {
      static public class C4 {
        static public int f;
      }
    }
  }
}
```

```
package second;

public class A {
  void m() {
    C1.C2.C3.C4.f = 1;
  }
}
```

Ex.6 - es Válido?

```
package first ;

public class C1 {
  static public class C2 {
    static private class C3 {
      static public class C4 {
        static public int f;
      }
    }
  }
}
```

```
package second;

public class A {
  void m() {
    C1.C2.C3.C4.f = 1;
  }
}
```

ERROR! C3 es privada

Ex.7 - es Válido?

```
package first ;

public class C1 {
  static public class C2 {
    static protected class C3 {
      static public class C4 {
        static public int f;
      }
    }
  }
}
```

```
package second;

public class A {
  void m() {
    C1.C2.C3.C4.f = 1;
  }
}
```

Ex.7 - es Válido?

```
package first ;

public class C1 {
  static public class C2 {
    static protected class C3 {
      static public class C4 {
        static public int f;
      }
    }
  }
}
```

```
package second;

public class A {
  void m() {
    C1.C2.C3.C4.f = 1;
  }
}
```

ERROR! C3 es protected pero están en distinto paquete

Ex.8 - es Válido?

```
package first ;

public class C1 {
  static public class C2 {
    static class C3 {
      static public class C4 {
        static public int f;
      }
    }
  }
}
```

```
package second;

public class A {
  void m() {
    C1.C2.C3.C4.f = 1;
  }
}
```

Ex.8 - es Válido?

```
package first ;

public class C1 {
  static public class C2 {
    static class C3 {
      static public class C4 {
        static public int f;
      }
    }
  }
}
```

```
package second;

public class A {
  void m() {
    C1.C2.C3.C4.f = 1;
  }
}
```

ERROR! C3 es package private y están en distinto paquete

Instancias de una clase interna

Si en una clase **A** ejecuta **new** $\mathbf{C}_1.\mathbf{C}_2.\mathbf{C}_3...\mathbf{C}_n$ entonces todas las clases $\mathbf{C}_1$ tienen que ser visibles a **A**

1. Si **A** y $\mathbf{C}_1$ están en paquetes distintos, entonces todas las clases $\mathbf{C}_i$ tienen que ser **public**.
2. Si **A** y $\mathbf{C}_1$ están en el mismo paquete, entonces todas las clases $\mathbf{C}_i$ tienen que ser **public, protected** o **package private**.

Instancias de una clase interna

Es similar a la creación de objetos de las clases externas. Únicamente hay que tener en cuenta que hay que cualificar la clase interna

```
public class A {
  public static int f1;
  public int f2;
  private int f3;

  static public class A1 {
    public static int g1;
    public int g2;
    private int g3;

    static public class A2 {
      public static int h1;
      public int h2;
      private int h3;
    }
  }
```

```
  void metodoClaseA(){
    A a = new A();
    a.f2 = 1;
    a.f3 = 2;
    A.f1 = 3;
    A.A1 a1 = new A.A1();
    a1.g2 = 4;
    a1.g3 = 5;
    A.A1.g1 = 6;
    a1.f2 = 7;
    a1.h2 = 8;
    A.A1.A2 a2 = new A.A1.A2();
    a2.h2 = 9;
    a2.h3 = 10;
    A.A1.A2.h1 = 11;
    a2.f2 = 13;
    a2.g2 = 14;
  }
}
```

Instancias de una clase interna

```
public class OuterClass { // paquete 1
  static class NestedClass1 {
    public static int f1;
    private int f4;
  }

  static class NestedClass2 {
    private static int f2;
    public int f3;
  }
}

public class OtherClass { // paquete1
  void foo() {
    OuterClass.NestedClass1 x = new OuterClass.NestedClass1();
    OuterClass.NestedClass1.NestedClass2 y = new OuterClass.NestedClass1.NestedClass2();
    OuterClass.NestedClass1.f1 = 3;
    x.f4 = 3;                    ?
    OuterClass.NestedClass1.NestedClass2.f2 =12;
    y.f3 = 15;
  }
}
```

Si se accede desde otra clase, tienen que ser visibles

Instancias de una clase interna

```
public class OuterClass { // paquete 1
  static class NestedClass1 {
    public static int f1;
    private int f4;
  }

  static class NestedClass2 {
    private static int f2;
    public int f3;
  }
}

public class OtherClass { // paquete1
  void foo() {
    OuterClass.NestedClass1 x = new OuterClass.NestedClass1();
    OuterClass.NestedClass1.NestedClass2 y = new OuterClass.NestedClass1.NestedClass2();
    OuterClass.NestedClass1.f1 = 3;
    x.f4 = 3;                    WRONG: f4 es privado
    OuterClass.NestedClass1.NestedClass2.f2 =12;        ?
    y.f3 = 15;
  }
}
```

Si se accede desde otra clase, tienen que ser visibles

Instancias de una clase interna

```
public class OuterClass { // paquete 1
  static class NestedClass1 {
    public static int f1;
    private int f4;
  }

  static class NestedClass2 {
    private static int f2;
    public int f3;
  }
}

public class OtherClass { // paquete1
  void foo() {
    OuterClass.NestedClass1 x = new OuterClass.NestedClass1();
    OuterClass.NestedClass1.NestedClass2 y = new OuterClass.NestedClass1.NestedClass2();
    OuterClass.NestedClass1.f1 = 3;
    x.f4 = 3;                    WRONG: f4 es privado
    OuterClass.NestedClass1.NestedClass2.f2 =12;          WRONG: f2 es privado
    y.f3 = 15;                   ?
  }
}
```

Si se accede desde otra clase, tienen que ser visibles

Instancias de una clase interna

```
public class OuterClass { // paquete 1
  static class NestedClass1 {
    public static int f1;
    private int f4;
  }

  static class NestedClass2 {
    private static int f2;
    public int f3;
  }
}

public class OtherClass { // paquete1
  void foo() {
    OuterClass.NestedClass1 x = new OuterClass.NestedClass1();
    OuterClass.NestedClass1.NestedClass2 y = new OuterClass.NestedClass1.NestedClass2();
    OuterClass.NestedClass1.f1 = 3;
    x.f4 = 3;                    WRONG: f4 es privado
    OuterClass.NestedClass1.NestedClass2.f2 =12;          WRONG: f2 es privado
    y.f3 = 15;                   OK! f3 es público
  }
}
```

Si se accede desde otra clase, tienen que ser visibles

Instancias de una clase interna

```
public class OuterClass { // paquete 1
  static class NestedClass1 {
    public static int f1;
    private int f4;
  }

  static class NestedClass2 {
    private static int f2;
    public int f3;
  }
}

public class OtherClass { // paquete2
  void foo() {
    OuterClass.NestedClass1 x = new OuterClass.NestedClass1();          ?
      OuterClass.NestedClass1.NestedClass2 y = new
      OuterClass.NestedClass1.NestedClass2();          ?
    OuterClass.NestedClass1.f1 = 3; ?
    x.f4 = 3; ?
    OuterClass.NestedClass1.NestedClass2.f2 =12; ?
    y.f3 = 15; ?
  }
}
```

Interfaces Anidadas

```
public class A {
  public interface B {
    public void f(C x);
    public void g(C x);
    public interface C {
      public int m();
    }
  }
}
```

Funciona igual que para el caso de las clases!!! No hace falta poner el static– se entiende que lo es por defecto

```
public class E implements A.B.C {
  @Override
  public int m() {...}
}
```

```
public class F implements A.B {
  @Override
  public int f(B.C x) {...}

  @Override
  public void g(B.C x) {...}
}
```

```
public class G extends A { }
```

Clases anidadas no estáticas (Inner Class)

```
class OuterClass {
  ...
  public class NestedClass {    //puede usarse protected, public, private o sin modificador
    ...
  }
}
```

Tiene algunas características muy útiles:

1. Se puede crear una instancia de **NestedClass** solo **usando una instancia** de **OuterClass** (inmediato)! Es **erróneo** (no compila) escribir:

   ```
   OuterClass.NestedClass a = new OuterClass.NestedClass(..);
   ```

   Lo correcto es:

   ```
   OuterClass.NestedClass a = (new OuterClass(...)).new NestedClass(..);
   ```

2. **NestedClass** "hereda" los atributos y métodos de **OuterClass**, es como extender **OuterClass** con otra funcionalidad!

Clases anidadas no estáticas (Inner Class)

```
public class OuterClass {
  private int f;

  public OuterClass(int n) { this.f = n; }

  public class InnerClass {
    public int getF() { return f; }
    public void setF(int n) { f = n; }
  }

  InnerClass foo() { return new InnerClass(); }
}

public static void main(String[] args) {
  OuterClass x = new OuterClass(1);
  OuterClass y = new OuterClass(1);
  OuterClass.InnerClass a = x.foo();
  OuterClass.InnerClass b = y.foo();
  OuterClass.InnerClass c = x.foo();
  a.setF(3);
  System.out.println ( a.getF() + " " + b.getF() + " " + c.getF());
}
```

Salida??

Clases anidadas no estáticas (Inner Class)

```
public class OuterClass {
  private int f;

  public OuterClass(int n) { this.f = n; }

  public class InnerClass {
    public int getF() { return f; }
    public void setF(int n) { f = n; }
  }

  InnerClass foo() { return new InnerClass(); }
}

public static void main(String[] args) {
  OuterClass x = new OuterClass(1);
  OuterClass y = new OuterClass(1);
  OuterClass.InnerClass a = x.foo();
  OuterClass.InnerClass b = y.foo();
  OuterClass.InnerClass c = x.foo();
  a.setF(3);
  System.out.println ( a.getF() + " " + b.getF() + " " + c.getF());
}
```

Salida??

3 1 3

Instancias de Inner Class

```
public class OuterClass {
  private int f;

  public OuterClass(int n) { this.f = n; }

  public class InnerClass {
    public int getF() { return f; }
    public void setF(int n) { f = n; }
  }

  InnerClass foo() { return new InnerClass(); }
}
```

Una instancia de **InnerClass** se puede crear solo usando una de **OuterClass**

Instancias de Inner Class

```
public class OuterClass {
  private int f;

  public OuterClass(int n) { this.f = n; }

  public class InnerClass {
    public int getF() { return f; }
    public void setF(int n) { f = n; }
  }

  InnerClass foo() { return new InnerClass(); }
}
```

Una instancia de **InnerClass** se puede crear solo usando una de **OuterClass**

```
public class SomeClass {
  OuterClass.InnerClass a = new OuterClass.InnerClass(); }
```

ERROR!!

Instancias de Inner Class

```
public class OuterClass {
  private int f;

  public OuterClass(int n) { this.f = n; }

  public class InnerClass {
    public int getF() { return f; }
    public void setF(int n) { f = n; }
  }

  InnerClass foo() { return new InnerClass(); }
}
```

Una instancia de **InnerClass** se puede crear solo usando una de **OuterClass**

```
public class SomeClass {
  OuterClass.InnerClass a = new OuterClass.InnerClass(); }
```

ERROR!!

```
public class SomeClass {
  OuterClass x = new OuterClass(1);
  OuterClass.InnerClass a = x.foo(); }
```

OK!

Instancias de Inner Class

```
public class OuterClass {
  private int f;

  public OuterClass(int n) { this.f = n; }

  public class InnerClass {
    public int getF() { return f; }
    public void setF(int n) { f = n; }
  }

  InnerClass foo() { return new InnerClass(); }
}
```

Una instancia de **InnerClass** se puede crear solo usando una de **OuterClass**

```
public class SomeClass {
  OuterClass.InnerClass a = new OuterClass.InnerClass(); }
```

ERROR!!

```
public class SomeClass {
  OuterClass x = new OuterClass(1);
  OuterClass.InnerClass a = x.foo(); }
```

OK!

```
public class SomeClass {
  OuterClass x = new OuterClass(1);
  OuterClass.InnerClass a = x. new InnerClass();}
```

OK!

Qué significa this?

Qué escribe este programa?

```
public class OuterClass {
  private int f;
  OuterClass(int m) {
    this.f = m;
  }

  public class InnerClass {
    private int f;


    InnerClass (int m) {
      this.f = m;
    }

    public void test() {
      System.out.println (this.f);
    }
  }
}
```

```
OuterClass x = new OuterClass(1);
InnerClass a = x.new InnerClass(4);
a.test();
```

Qué significa this?

Qué escribe este programa?

```
public class OuterClass {
  private int f;
  OuterClass(int m) {
    this.f = m;
  }

  public class InnerClass {
    private int f;


    InnerClass (int m) {
      this.f = m;
    }

    public void test() {
      System.out.println (this.f);
    }
  }
}
```

```
OuterClass x = new OuterClass(1);
InnerClass a = x.new InnerClass(4);
a.test();
```

4

Qué significa this?

Qué escribe este programa?

```
public class OuterClass {
  private int f;
  OuterClass(int m) {
    this.f = m;
  }

  public class InnerClass {
    private int f;

    InnerClass (int m) {
      this.f = m;
    }
    public void test() {
      System.out.println (this.f + " " +OuterClass.this.f);
    }
  }
}
```

```
OuterClass x = new OuterClass(1);
InnerClass a = x.new InnerClass(4);
a.test();
```

Qué significa this?

Qué escribe este programa?

```
public class OuterClass {
  private int f;
  OuterClass(int m) {
    this.f = m;
  }

  public class InnerClass {
    private int f;

    InnerClass (int m) {
      this.f = m;
    }
    public void test() {
      System.out.println (this.f + " " +OuterClass.this.f);
    }
  }
}
```

```
OuterClass x = new OuterClass(1);
InnerClass a = x.new InnerClass(4);
a.test();
```

4 1

Qué significa this?

Qué escribe este programa?

```
public class OuterClass {
  private int f = 1;

  public class InnerClass1 {
    private int f = 2;

    public class InnerClass2 {
      private int f = 3;
      public void test() {
        System.out.println(this.f + " " + InnerClass1.this.f);
      }
    }
    public void test() {
      System.out.println(this.f + " " + OuterClass.this.f);
    }
  }
}
```

```
OuterClass x = new OuterClass();
OuterClass.InnerClass1 a = x.new InnerClass1();
OuterClass.InnerClass1.InnerClass2 b = a.new InnerClass2();
a.test();
b.test();
```

Qué significa this?

Qué escribe este programa?

```
public class OuterClass {
  private int f = 1;

  public class InnerClass1 {
    private int f = 2;

    public class InnerClass2 {
      private int f = 3;
      public void test() {
        System.out.println(this.f + " " + InnerClass1.this.f);
      }
    }
    public void test() {
      System.out.println(this.f + " " + OuterClass.this.f);
    }
  }
}
```

```
OuterClass x = new OuterClass();
OuterClass.InnerClass1 a = x.new InnerClass1();
OuterClass.InnerClass1.InnerClass2 b = a.new InnerClass2();
a.test();
b.test();
```

2 1

3 2

Qué significa this?

Qué escribe este programa?

```
public class ShadowTest {
  public int x = 0;
  class FirstLevel {
    public int x = 1;

    void methodInFirstLevel(int x) {
      System.out.println ("x = " + x);
      System.out.println ("this.x = " + this.x);
      System.out.println ("ShadowTest.this.x = " + ShadowTest.this.x);
    }
  }

  public static void main(String[] args) {
    ShadowTest st = new ShadowTest();
    ShadowTest.FirstLevel fl = st.new FirstLevel();
    fl.methodInFirstLevel(23);
  }
}
```

Qué significa this?

Qué escribe este programa?

```
public class ShadowTest {
  public int x = 0;
  class FirstLevel {
    public int x = 1;

    void methodInFirstLevel(int x) {
      System.out.println ("x = " + x);
      System.out.println ("this.x = " + this.x);
      System.out.println ("ShadowTest.this.x = " + ShadowTest.this.x);
    }
  }

  public static void main(String[] args) {
    ShadowTest st = new ShadowTest();
    ShadowTest.FirstLevel fl = st.new FirstLevel ();
    fl.methodInFirstLevel(23);
  }
}
```

23

1

0

Ex.1: Es Válido?

```
package first ;

public class OuterClass {
  private class InnerClass {
    ...
  }
  ...
}
```

```
package first ;

public class SomeClass {
  ...
  OuterClass x = new OuterClass(1);
  OuterClass.InnerClass a = x.new InnerClass();
  ....
}
```

Ex.1: Es Válido?

```
package first ;

public class OuterClass {
  private class InnerClass {
    ...
  }
  ...
}
```

```
package first ;

public class SomeClass {
  ...
  OuterClass x = new OuterClass(1);
  OuterClass.InnerClass a = x.new InnerClass();
  ....
}
```

NO!

InnerClass es privada

Ex.2: Es Válido?

```
package first ;

public class OuterClass {
  protected class InnerClass {
    ...
  }
  ...
}
```

```
package first ;

public class SomeClass {
  ...
  OuterClass x = new OuterClass(1);
  OuterClass.InnerClass a = x.new InnerClass();
  ....
}
```

Ex.2: Es Válido?

```
package first ;

public class OuterClass {
  protected class InnerClass {
    ...
  }
  ...
}
```

```
package first ;

public class SomeClass {
  ...
  OuterClass x = new OuterClass(1);
  OuterClass.InnerClass a = x.new InnerClass();
  ....
}
```

OK!

InnerClass es protected y en el mismo paquete

Ex.3: Es Válido?

```
package first ;

public class OuterClass {
  class InnerClass {
    ...
  }
  ...
}
```

```
package first ;

public class SomeClass {
  ...
  OuterClass x = new OuterClass(1);
  OuterClass.InnerClass a = x.new InnerClass();
  ....
}
```

Ex.3: Es Válido?

```
package first ;

public class OuterClass {
  class InnerClass {
    ...
  }
  ...
}
```

```
package first ;

public class SomeClass {
  ...
  OuterClass x = new OuterClass(1);
  OuterClass.InnerClass a = x.new InnerClass();
  ....
}
```

OK!

InnerClass es package protected y está en el mismo paquete

Ex.4: Es Válido?

```
package first ;

public class OuterClass {
  class InnerClass {
    ...
  }
  ...
}
```

```
package second;

public class SomeClass {
  ...
  OuterClass x = new OuterClass(1);
  OuterClass.InnerClass a = x.new InnerClass();
  ....
}
```

Ex.4: Es Válido?

```
package first ;

public class OuterClass {
  class InnerClass {
    ...
  }
  ...
}
```

```
package second;

public class SomeClass {
  ...
  OuterClass x = new OuterClass(1);
  OuterClass.InnerClass a = x.new InnerClass();
  ....
}
```

ERROR!

Distintos paquetes

Clases Locales

```
class OuterClass {
  ...
  public Iterator <Integer> m() {
    class MyLocalCLass implements Iterator<Integer> {
      ...
    }
    return new MyLocalClass();
  }
}
```

- Son clases **internas** pero que se definen dentro de un bloque de código o método. La clase solo se puede utilizar en dicho bloque.
- Normalmente implementan una interfaz pública (o heredan de una clase pública) para poder devolverlas a otra clase, etc.
- Una clase local puede acceder de la clase contenedora en la que se encuentra.

Clases Locales

```
class OuterClass {
  ...
  Iterator <Integer> m(int i, final int j) {
    final int k = 5;
    int h;
    class MyLocalCLass implement Iterator<Integer> {
      ...
      a = i; // ERROR!
      b = j; // OK!
      b = k; // OK!
      b = h; // ERROR!
    }
    return new MyLocalCLass();
  }
```

- Una clase local tiene acceso a las variables locales y parámetros (en caso de estar definida en un método) del bloque en el que se encuentra. Para ello es obligatorio que dichas variables locales o parámetros sean efectivamente **finales**, es decir, llevan el modificador **final**, o bien nunca se modifican

Clases Anónimas

```
class OuterClass {
  ...
  Iterator <Integer> m() {
    return new Iterator <Integer>() {
      public boolean hasNext() {...}
      public Integer next() {...}
      public void remove() {...}
    };
  }
}
```

- Son como clases locales pero **sin nombre**.
- Normalmente implementan una interfaz pública o heredan de otra clase pública (si no serían inútiles).
- **No pueden tener constructoras.**
- Su mayor utilidad es usarlas en **Swing** para implementar los **Listeners**.

Clases Anónimas

```
public class ButtonExample implements ActionListener {
  private JButton redButton;
  public JPanel createContentPane() {
    redButton = new JButton("Red Score!");
    redButton.addActionListener(this);
    ...
  }

  @Override
  public void actionPerformed(ActionEvent e) {
    if (e.getSource()==redButton) ...
  }
}
```

- Cuando se hace un click sobre el botón (se registra en el método addActionListener), se llama al método **actionPerformed** del objeto this.
- **actionPerformed** es un método de ActionListener (interfaz que implementa ButtonExample).

Clases Anónimas

Cómo implementamos un botón utilizando una clase anónima?

```
class ButtonExample {
  public JPanel createContentPane() {
    redButton = new JButton("Red Score!");
    redButton.addActionListner( new ActionListener() {
      public void actionPerformed(ActionEvent e) {
        // do something
      }
    });
  }
```

- addActionListener no se llama sobre this.
- accActionListener se llama sobre una clase anónima que implementa la interfaz ActionListener (fijaos donde se cierra el paréntesis).

Otros usos de clases anónimas

```
public class A {
  public void f() { g();}
  public void g() {
    System.out.println ("Hola");
  }
}

public class B {
  public void q() {
    System.out.println ("Hola Hola");
  }
  public A h() {
    return new A() {
      public void g() { q(); }
    };
  }
}

public class Main {
  public static void main(String[] args) {
    B x = new B();
    A y = x.h();
    y.f();
  }
}
```

- Qué imprime por pantalla?

Otros usos de clases anónimas

```
public class A {
  public void f() { g();}
  public void g() {
    System.out.println ("Hola");
  }
}

public class B {
  public void q() {
    System.out.println ("Hola Hola");
  }
  public A h() {
    return new A() {
      public void g() { q(); }
    };
  }
}

public class Main {
  public static void main(String[] args) {
    B x = new B();
    A y = x.h();
    y.f();
  }
}
```

Hola Hola

- Qué imprime por pantalla?

Ejemplo 2

```
public class MyList<T> implements Iterable<T> {
  private final static int _INIT_SIZE = 2;
  protected T[] elem;
  protected int size ;

  public MyList() {...}
  public void addElem(T e) {...}
  public T getElem(int index) {...}
  public int size () { return size ; }

  @Override
  public Iterator <T> iterator() {
    return MyIterator<T>(this);
  }
}
```

```
public class MyIterator<T> implements
      Iterator<T> {

  private int currentPos = -1;
  private MyList<T> lista;

  public MyIterator(MyList<T> l){
    this.lista = l;
  }
  @Override
  public boolean hasNext() {
    return currentPos < lista.size -1;
  }
  @Override
  public T next() {
    currentPos++;
    return lista.elem[currentPos];
  }
}
```

Ejemplo 2

```
public class MyList<T> implements Iterable<T> {

  //todo igual que antes

  public Iterator <T> iterator() {
    return new Iterator <T>(){
      private int currentPos = -1;
      @Override
      public boolean hasNext() {
        return this.currentPos < size - 1;
      }
      @Override
      public T next() {
        this.currentPos++;
        return elements[currentPos];
      }
    };
  }
}
```

Ejemplo 2

```
public class MyList<T> implements Iterable<T> {

  //todo igual que antes

  public Iterator <T> iterator() {
    return new Iterator <T>(){
      private int currentPos = -1;
      @Override
      public boolean hasNext() {
        return this.currentPos < size - 1;
      }
      @Override
      public T next() {
        this.currentPos++;
        return elements[currentPos];
      }
    };
  }
}
```

- Nos evitamos pasar **MyList** al Iterador, ya que al ser clase local tiene acceso a todos los atributos.